

NOUVELAIR

Cahier des Charges Detaille

Systeme de Gestion de Reservations Aeriennes

Application Web Django 4.2 | Formation Test / QA

Version 1.0 - Avril 2026

Document confidentiel - Usage interne et formation

Table des Matieres

1. Introduction	5
1.1 Contexte du Projet	5
1.2 Objectifs du Document	5
1.3 Portee du Projet	5
1.4 Technologies Utilisees	6
2. Specifications Fonctionnelles	6
2.1 Module Gestion des Vols (flights)	6
2.1.1 Modeles de Donnees	6
2.1.2 Vues et Fonctionnalites	8
2.1.3 URL Patterns	8
2.2 Module Reservations (bookings)	9
2.2.1 Modeles de Donnees	9
2.2.2 Vues et Fonctionnalites	9
2.3 Module Comptes Utilisateurs (accounts)	10
2.3.1 Modele UserProfile	10
2.3.2 Vues et Fonctionnalites	10
2.4 Module Destinations (destinations)	10
2.5 Module Promotions (promotions)	11
3. Architecture Technique	11

3.1 Structure du Projet	11
3.2 Configuration Django (settings.py)	12
3.3 Context Processor	12
3.4 Structure des Templates	12
4. Schema de la Base de Donnees	13
4.1 Donnees de Test	13
4.2 Aeroports dans la Base	13
5. Exigences Non-Fonctionnelles	14
5.1 Securite	14
5.2 Performance	14
5.3 Compatibilite et Responsive Design	14
5.4 Internationalisation	15
6. Cartographie Complete des URLs	15
6.1 Module Flights	15
6.2 Module Bookings	15
6.3 Module Accounts	16
6.4 Module Destinations	16
6.5 Module Promotions	16
6.6 Pages Legales (TemplateView)	16
7. Strategie de Test et Assurance Qualite	17

7.1 Tests Unitaires	17
7.2 Tests d'Integration	17
7.3 Tests End-to-End avec IA	17
8. Guide de Deploiement et Installation	17
8.1 Prerequis	18
8.2 Procedure d'Installation	18
8.3 Comptes de Test	18
9. Problemes Connus et Solutions	19
9.1 Erreurs de Namespace URL	19
9.2 Champs de Modele	19
9.3 Filtres de Template	19
9.4 Pages Legales Manquantes	19
10. Glossaire	20

1. Introduction

1.1 Contexte du Projet

NouvelAir est une compagnie aerienne tunisienne qui souhaite moderniser son systeme de gestion de reservations. Actuellement, la gestion des vols, des reservations et des clients est effectuee de maniere partiellement manuelle, ce qui entraine des problemes d'efficacite, des erreurs de saisie et une experience client sous-optimale. Le projet consiste a developper une application web complete basee sur le framework Django 4.2 qui couvrira l'ensemble du cycle de reservation, depuis la recherche de vols jusqu'a la confirmation et la gestion post-reservation.

Ce projet a egalement une vocation pedagogique : il sert de support pour la formation en test et assurance qualite (QA). L'application a ete concue avec une couverture de tests unitaires et d'integration, ainsi qu'un module de test assiste par intelligence artificielle, permettant aux apprenants de se familiariser avec les bonnes pratiques de test dans un environnement Django realiste.

1.2 Objectifs du Document

Ce cahier des charges a pour objectif de definir de maniere exhaustive et structuree les specifications fonctionnelles et techniques du systeme NouvelAir. Il servira de reference tout au long du cycle de developpement et sera utilise comme base pour la planification des sprints, la redaction des cas de test et la validation des livrables. Le document couvre les aspects suivants : les besoins fonctionnels detailles pour chaque module, l'architecture technique, le schema de la base de donnees, la structure des URLs, les exigences non-fonctionnelles, et la strategie de test.

1.3 Portee du Projet

Le systeme NouvelAir est decoupe en cinq modules principaux, chacun correspondant a une application Django distincte. Cette architecture modulaire facilite la maintenance, les evolutions futures et les tests isolés de chaque composant. Les cinq modules sont : Gestion des Vols (flights), Gestion des Reservations (bookings), Gestion des Comptes Utilisateurs (accounts), Gestion des Destinations (destinations) et Gestion des Promotions (promotions). Chaque module dispose de ses propres modeles, vues, formulaires, templates et tests unitaires.

Module	Application Django	Responsabilite principale
Gestion des Vols	flights	Recherche, affichage et gestion des vols et aeroports
Reservations	bookings	Creation, suivi et annulation des reservations

Comptes	accounts	Inscription, connexion et profils utilisateurs
Destinations	destinations	Pages d'information sur les destinations
Promotions	promotions	Codes promo et offres speciales

Tableau 1 : Recapitulatif des modules de l'application NouvelAir

1.4 Technologies Utilisees

Le choix technologique s'est porte sur un stack Python/Django reconnu pour sa robustesse, sa securite et sa communaute active. Les technologies suivantes sont utilisees dans le projet :

Technologie	Version	Usage
Python	3.12.4	Langage principal
Django	4.2.30	Framework web full-stack
SQLite	3	Base de donnees (developpement)
Bootstrap 5	5.3	Framework CSS responsive
crispy-forms	2.x	Rendu elegante des formulaires Django
django-countries	7.x	Champs de pays normalises
django-phonenumbers-field	7.x	Validation des numeros de telephone
Pillow	10.x	Traitement des images (avatars, photos)

2. Specifications Fonctionnelles

2.1 Module Gestion des Vols (flights)

Le module Gestion des Vols est le coeur fonctionnel de l'application. Il permet aux utilisateurs de rechercher des vols disponibles, de consulter les details de chaque vol et de parcourir la liste des aeroports desservis. Ce module interagit etroitement avec le module de reservations, car la recherche de vols constitue la premiere etape du parcours de reservation.

2.1.1 Modeles de Donnees

Le module flights definit quatre modeles principaux qui structurent l'ensemble des donnees liees aux operations aeriennes. Ces modeles sont concus pour refléter fidèlement la realite operationnelle d'une compagnie aérienne.

Modele Airport : Represente un aeroport avec ses coordonnees geographiques. Chaque aeroport est identifie de maniere unique par un code IATA (par exemple TUN pour Tunis-Carthage, CDG pour Paris-Charles de Gaulle). Les champs incluent le code (CharField, max 3 caracteres), le nom complet, la ville, le pays, ainsi que la latitude et la longitude en FloatField pour le positionnement GPS. La base de donnees est peulee avec 10 aeroports couvrant la Tunisie et les principales destinations europeennes et moyen-orientales.

Modele Aircraft : Decrit chaque appareil de la flotte. Les champs principaux sont model_name (designation commerciale de l'appareil), registration (immatriculation unique de l'avion), total_seats, economy_seats, business_seats (repartition des places par classe) et is_active (indique si l'appareil est actuellement en service). La flotte comprend 4 appareils de types differents pour simuler une flotte reelle.

Modele Flight : Represente un vol specifique avec ses caracteristiques operationnelles. Les champs incluent flight_number (numero de vol unique), origin et destination (cles etrangeres vers Airport), aircraft (cle etrangere vers Aircraft), departure_time et arrival_time (DateTimeField), base_price_economy et base_price_business (DecimalField pour les tarifs de base), available_seats_economy et available_seats_business (IntegerField pour le suivi des places disponibles), status (choix : scheduled, boarding, in_flight, landed, cancelled) et une methode get_current_price_economy qui calcule le prix dynamique. Le modele inclut egalement une methode get_duration_display qui formate la duree du vol de maniere lisible. La base contient 284 vols repartis sur plusieurs semaines.

Modele FlightPriceHistory : Enregistre l'historique des variations de prix pour chaque vol, permettant de tracer les evolutions tarifaires dans le temps.

Detail des champs du modele Flight :

Champ	Type	Description
flight_number	CharField	Numero unique du vol (ex: NU201)
origin	FK -> Airport	Aeroport de depart
destination	FK -> Airport	Aeroport d'arrivee
aircraft	FK -> Aircraft	Appareil affecte au vol
departure_time	DateTimeField	Heure de depart (timezone-aware)
arrival_time	DateTimeField	Heure d'arrivee (timezone-aware)
base_price_economy	DecimalField	Prix de base classe economie (TND)
base_price_business	DecimalField	Prix de base classe affaires (TND)

available_seats_economy	IntegerField	Places disponibles economie
available_seats_business	IntegerField	Places disponibles affaires
status	CharField (choices)	scheduled / boarding / in_flight / landed / cancelled

2.1.2 Vues et Fonctionnalités

Le module flights implemente cinq vues principales, chacune correspondant a une fonctionnalité spécifique de l'interface utilisateur :

HomeView (page d'accueil) : Vue principale de l'application. Elle affiche le formulaire de recherche de vols (origine, destination, date), les vols populaires ou mis en avant, les destinations recommandées et les promotions actives. C'est le point d'entrée principal de l'application. L'URL correspondante est '/' avec le nom 'home' dans l'espace de noms 'flights'.

FlightSearchView (recherche de vols) : Traite les requêtes de recherche émises depuis le formulaire d'accueil. Elle accepte les paramètres d'origine, de destination et de date, puis filtre les vols disponibles dans la base de données. Les résultats sont affichés avec les informations essentielles : numéro de vol, trajet, horaires, durée, prix et nombre de places restantes. Un mécanisme d'autocomplétion est disponible pour les champs d'aéroports via l'endpoint `airport_autocomplete`. L'URL est '/search/' avec le nom 'flight_search'.

FlightDetailView (detail d'un vol) : Affiche les informations complètes d'un vol spécifique identifié par son identifiant. L'utilisateur peut consulter les détails de l'appareil, les horaires exacts, les tarifs par classe et procéder à la réservation directement depuis cette page. L'URL utilise le pattern '/flight/<int:pk>/' avec le nom 'flight_detail'.

AirportListView (liste des aéroports) : Présente la liste complète des aéroports desservis par NouvelAir. Chaque aéroport est affiché avec son code IATA, sa localisation géographique et un lien vers sa page de détail. L'URL est '/airports/' avec le nom 'airport_list'.

AirportDetailView (detail d'un aéroport) : Affiche les informations détaillées d'un aéroport, y compris sa localisation GPS et la liste des vols au départ et à l'arrivée. L'URL utilise le pattern '/airport/<int:pk>/' avec le nom 'airport_detail'.

2.1.3 URL Patterns

Toutes les URLs du module flights sont préfixées par l'espace de noms 'flights'. Dans les templates, les URLs doivent impérativement être référencées avec la syntaxe `{% url 'flights:home' %}` et non `{% url 'home' %}` uniquement, sinon une erreur `NoReverseMatch` sera levée. Voici le mapping complet des URLs :

Pattern URL	Nom (name)	Vue associée
-------------	------------	--------------

/	home	HomeView
/search/	flight_search	FlightSearchView
/flight/<int:pk>/	flight_detail	FlightDetailView
/airports/	airport_list	AirportListView
/airport/<int:pk>/	airport_detail	AirportDetailView

2.2 Module Reservations (bookings)

Le module Reservations gere l'ensemble du cycle de vie d'une reservation, depuis sa creation par un utilisateur authentifie jusqu'a son annulation eventuelle. Ce module est le plus critique sur le plan fonctionnel car il implique des transactions financieres et doit garantir la coherence des donnees (places disponibles, montants, statuts). La conception des modeles et des vues prend en compte les contraintes de concurrence pour eviter les sur-reservations.

2.2.1 Modeles de Donnees

Modele Booking : Represente une reservation de vol. Les champs principaux incluent : booking_reference (chaîne unique de reference), user (cle etrangere vers le modele User de Django), flight (cle etrangere vers Flight), passenger_first_name, passenger_last_name (noms du passager), passenger_email, passenger_phone, travel_class (choix : economy, business), number_of_passengers (IntegerField), status (choix : pending, confirmed, cancelled, completed), total_price (DecimalField), et les timestamps created_at et updated_at.

Modele Passenger : Stocke les informations detaillees de chaque passager associe a une reservation. Ce modele est lie a Booking par une relation many-to-one. Les champs incluent passport_number, date_of_birth, nationality, gender et special_needs.

2.2.2 Vues et Fonctionnalites

CreateBookingView : Formulaire de creation de reservation accessible uniquement aux utilisateurs authentifies. Le formulaire collecte les informations du passager, la classe de voyage et le nombre de passagers. Apres validation, la reservation est creee avec le statut 'pending' et une reference unique est generee. URL : '/bookings/create/' (name='create').

BookingDetailView : Affiche le detail complet d'une reservation specifique. L'utilisateur peut voir les informations du vol, les coordonnees du passager, le montant total et le statut actuel. Un bouton d'annulation est disponible si le statut le permet. URL : '/bookings/<int:pk>/' (name='detail').

BookingListView : Liste toutes les reservations de l'utilisateur connecte. URL : '/bookings/' (name='booking_list').

BookingLookupView : Permet a un utilisateur non authentifie de rechercher une reservation via sa reference de booking et son nom de famille. URL : '/bookings/lookup/' (name='lookup').

CancelBookingView : Permet l'annulation d'une reservation si les conditions le permettent (delai avant le depart, statut actuel). URL : '/bookings/<int:pk>/cancel/' (name='cancel').

2.3 Module Comptes Utilisateurs (accounts)

Le module accounts gere l'authentification et les profils utilisateurs. Il utilise le systeme d'authentification natif de Django (AUTH_USER_MODEL='auth.User') associe a un modele UserProfile complementaire pour stocker les informations personnelles etendues. Ce choix architectural permet de beneficier de la robustesse du systeme de Django tout en ajoutant des champs specifiques au contexte d'une compagnie aerienne.

2.3.1 Modele UserProfile

Le modele UserProfile est lie a User par une relation OneToOne. Il stocke les informations personnelles avancees suivantes : address (adresse postale), avatar (ImageField pour la photo de profil), city et country, date_of_birth (DateField), gender (choix), nationality (pays via django-countries), newsletter (BooleanField pour le consentement marketing), passport_number (numero de passeport), phone (PhoneNumberField), ainsi que les timestamps created_at et updated_at. Le chargement des fixtures de test cree deux utilisateurs : admin (superutilisateur) et testuser, chacun avec un profil complet.

2.3.2 Vues et Fonctionnalites

Le module implemente les vues classiques d'un systeme d'authentification : LoginView (connexion via formulaire username/password avec redirection), LogoutView (deconnexion avec redirection vers l'accueil), RegisterView (inscription avec creation de User + UserProfile), et ProfileView (affichage et modification du profil). Les URL utilisent le prefixe '/accounts/' et l'espace de noms 'accounts'. Les identifiants de test sont : admin/NouvelAir2025! et testuser/NouvelAir2025!.

2.4 Module Destinations (destinations)

Le module destinations presente les informations touristiques et pratiques sur les differentes destinations desservies par NouvelAir. Il s'agit d'un module principalement informatif qui enrichit l'experience utilisateur en fournissant des details sur chaque ville ou pays accessible via les vols de la compagnie. Ce module contribue egalement au referencement SEO de l'application en proposant des pages descriptives riches en contenu pour chaque destination.

Modele Destination : Les champs incluent name, slug (pour les URLs lisibles), short_description et description (texte complet), category (choix : beach, culture, adventure, business), rating (DecimalField pour la note moyenne), is_featured (BooleanField pour la mise en avant), image

(ImageField) et airport (cle etrangere optionnelle vers Airport). La base contient 5 destinations de types varies. Les vues incluent DestinationListView ('/destinations/', name='destination_list') et DestinationDetailView ('/destinations/<slug:slug>', name='detail').

2.5 Module Promotions (promotions)

Le module promotions gere les offres commerciales et codes de reduction proposes par NouvelAir. Ce module permet de stimuler les ventes en offrant des reductions temporelles et cibles. L'implementation inclut la validation automatique des codes promo lors de la creation de reservation (verification de la validite, de la date d'expiration et du statut actif).

Modele Promotion : Les champs incluent code (chaine unique identifiant la promotion), description, discount_percentage (pourcentage de reduction), start_date et end_date (dates de validite) et is_active (BooleanField). La base contient 3 promotions types. Les vues incluent PromotionListView ('/promotions/', name='promotion_list') et PromotionDetailView ('/promotions/<int:pk>', name='detail').

3. Architecture Technique

3.1 Structure du Projet

Le projet suit la structure standard d'un projet Django 4.2 avec le pattern projet/applications. Le repertoire principal nouvelair_project contient le fichier de configuration manage.py et le sous-repertoire nouvelair/ qui regroupe les parametres globaux du projet. Chaque module fonctionnel est implemente comme une application Django independante dans son propre sous-repertoire au meme niveau que le package du projet.

Repertoire	Contenu
nouvelair_project/	Racine du projet Django
nouvelair/	Package de configuration (settings, urls, wsgi)
flights/	Application Gestion des Vols
bookings/	Application Reservations
accounts/	Application Comptes Utilisateurs
destinations/	Application Destinations
promotions/	Application Promotions
ai_testing/	Module de test assiste par IA

scripts/	Scripts utilitaires (populate_test_data.py)
fixtures/	Donnees initiales (initial_data.json)

3.2 Configuration Django (settings.py)

Le fichier settings.py configure les parametres essentiels du projet. Les configurations critiques sont les suivantes :

Parametre	Valeur	Description
AUTH_USER_MODEL	'auth.User'	Utilise le modele User natif de Django
USE_TZ	True	Support des fuseaux horaires
TIME_ZONE	'Africa/Tunis'	Fuseau horaire de reference (Tunisie)
CURRENCY	'TND'	Devise utilisee (Dinar Tunisien)
LANGUAGE_CODE	'fr'	Langue par default (français)
INSTALLED_APPS	(voir liste)	5 apps + crispy_forms + countries + phonenumbers
TEMPLATES_DIRS	[BASE_DIR / 'templates']	Repertoire des templates globaux
LOGIN_URL	'accounts:login'	URL de redirection pour authentification
LOGIN_REDIRECT_URL	'flights:home'	URL apres connexion reussie
MEDIA_ROOT / URL	BASE_DIR / 'media'	Stockage des fichiers uploads

3.3 Context Processor

Un processeur de contexte personnalise est defini dans nouvelair/context_processors.py. Il injecte automatiquement les variables site_name (nom du site) et site_tagline (slogan) dans tous les templates, permettant une personnalisation centralisee de l'interface sans modifier chaque vue individuellement. Ce processeur est enregistre dans la configuration TEMPLATES['OPTIONS']['context_processors'].

3.4 Structure des Templates

Le systeme de templates utilise le mecanisme d'heritage de Django. Le template de base (nouvelair/templates/base.html) definit la structure commune de toutes les pages : barre de

navigation (navbar) avec les liens principaux, zone de contenu principal ({% block content %}) et pied de page (footer) avec les liens legaux, le formulaire de newsletter et les informations de contact. Chaque application possede son propre sous-repertoire de templates pour ses pages specifiques.

Navigation : La navbar contient les liens suivants avec les espaces de noms correspondants : Accueil (flights:home), Vols (flights:flight_search), Destinations (destinations:destination_list), Promotions (promotions:promotion_list), Ma Reservation (bookings:booking_list, visible si connecte), Login/Logout/Register (accounts:login, accounts:logout, accounts:register). Les URLs dans les templates doivent toujours utiliser la syntaxe {% url 'namespace:name' %} pour eviter les erreurs NoReverseMatch.

Pied de page : Le footer contient trois colonnes : Acces rapide (liens vers les pages principales), Assistance (liens vers legal.html et terms.html pour les mentions legales et conditions generales), et un formulaire d'inscription a la newsletter. Les pages legales et conditions sont servies par des TemplateView directes definies dans nouvelair/urls.py.

4. Schema de la Base de Donnees

La base de donnees SQLite est utilisee pour le developpement et les tests. Elle est peuplee automatiquement via la commande de gestion Django `populate_data` (flights/management/commands/populate_data.py). Les donnees de test sont suffisamment riches pour couvrir la plupart des cas d'utilisation et des cas de test.

4.1 Donnees de Test

Entite	Quantite	Details
Aeroports (Airport)	10	Tunisie + Europe + Moyen-Orient
Appareils (Aircraft)	4	Differents types d'avions
Vols (Flight)	284	Repartition sur plusieurs semaines
Destinations	5	Beach, Culture, Adventure, Business
Promotions	3	Codes de reduction actifs
Utilisateurs	2	admin + testuser avec profils complets

4.2 Aeroports dans la Base

Code	Nom	Ville	Pays
------	-----	-------	------

TUN	Tunis-Carthage	Tunis	Tunisie
MIR	Habib Bourguiba	Monastir	Tunisie
DJE	Zarzis	Djerba	Tunisie
SFA	Sfax-Thyna	Sfax	Tunisie
TOE	Nefta	Tozeur	Tunisie
CDG	Charles de Gaulle	Paris	France
FCO	Leonardo da Vinci	Rome	Italie
IST	Istanbul Airport	Istanbul	Turquie
CMN	Mohammed V	Casablanca	Maroc
ALG	Houari Boumediene	Alger	Algerie

5. Exigences Non-Fonctionnelles

5.1 Securite

La securite est un aspect fondamental du systeme NouvelAir, particulierement dans le contexte d'une application gerant des donnees personnelles et des transactions financieres. Les mesures de securite implementees comprennent : la protection CSRF de Django (automatique via le middleware `CsrfViewMiddleware`), le hachage des mots de passe avec l'algorithme PBKDF2 de Django, la protection contre les injections SQL via l'ORM Django, la validation des donnees en amont par les formulaires Django et crispy-forms, et la gestion des sessions securisees via le framework d'authentification de Django. Les vues sensibles (creation de reservation, profil utilisateur) sont protegees par le decorateur `@login_required`.

5.2 Performance

L'application doit repondre aux exigences de performance suivantes : temps de chargement des pages inferieur a 3 secondes pour les pages statiques et inferieur a 5 secondes pour les pages avec requetes base de donnees complexes (recherche de vols). L'autocompletion des aeroports doit repondre en moins de 500 millisecondes. Les requetes de recherche doivent utiliser les index de base de donnees de maniere optimale, notamment sur les champs `origin`, `destination` et `departure_time` du modele `Flight`.

5.3 Compatibilite et Responsive Design

L'interface utilisateur est conçue avec Bootstrap 5 pour garantir une compatibilité maximale avec les navigateurs modernes (Chrome, Firefox, Safari, Edge) et un design responsive s'adaptant aux différentes tailles d'écran (desktop, tablette, mobile). Les points de rupture Bootstrap standards sont utilisés : 576px (sm), 768px (md), 992px (lg) et 1200px (xl). Le framework crispy-forms avec le template crispy_bootstrap5 assure un rendu cohérent des formulaires sur tous les périphériques.

5.4 Internationalisation

Le projet est initialement développé en français (LANGUAGE_CODE='fr') avec le fuseau horaire de la Tunisie (Africa/Tunis). La monnaie utilisée est le Dinar Tunisien (TND). L'architecture du projet permet une future internationalisation via le système i18n de Django si nécessaire, bien que cette fonctionnalité ne soit pas implémentée dans la version actuelle. Les noms de pays sont gérés par la bibliothèque django-countries qui supporte les noms en plusieurs langues.

6. Cartographie Complete des URLs

Le tableau ci-dessous présente la cartographie complète de toutes les URLs de l'application. Il est essentiel de respecter les espaces de noms (app_name) lors de la création ou la modification des templates pour éviter les erreurs de résolution URL. Chaque application Django définit son propre app_name dans son fichier urls.py.

6.1 Module Flights

URL	Nom	Vue	Methode
/	flights:home	HomeView	GET
/search/	flights:flight_search	FlightSearchView	GET/POST
/flight/<int:pk>/	flights:flight_detail	FlightDetailView	GET
/airports/	flights:airport_list	AirportListView	GET
/airport/<int:pk>/	flights:airport_detail	AirportDetailView	GET

6.2 Module Bookings

URL	Nom	Vue	Methode
/bookings/create/	bookings:create	CreateBookingView	GET/POST

/bookings/<int:pk>/	bookings:detail	BookingDetailView	GET
/bookings/	bookings:booking_list	BookingListView	GET
/bookings/lookup/	bookings:lookup	BookingLookupView	GET/POST
/bookings/<int:pk>/cancel/	bookings:cancel	CancelBookingView	POST

6.3 Module Accounts

URL	Nom	Vue	Methode
/accounts/login/	accounts:login	LoginView	GET/POST
/accounts/logout/	accounts:logout	LogoutView	GET
/accounts/register/	accounts:register	RegisterView	GET/POST
/accounts/profile/	accounts:profile	ProfileView	GET

6.4 Module Destinations

URL	Nom	Vue
/destinations/	destinations:destination_list	DestinationListView
/destinations/<slug:slug>/	destinations:detail	DestinationDetailView

6.5 Module Promotions

URL	Nom	Vue
/promotions/	promotions:promotion_list	PromotionListView
/promotions/<int:pk>/	promotions:detail	PromotionDetailView

6.6 Pages Legales (TemplateView)

Les pages legales sont definies directement dans nouvelair/urls.py en utilisant des TemplateView generique de Django, sans espace de noms. Les templates correspondants (legal.html et terms.html) sont stockes dans le repertoire nouvelair/templates/.

URL	Nom	Template
/legal/	legal	legal.html
/terms/	terms	terms.html

7. Strategie de Test et Assurance Qualite

Le projet NouvelAir integre une strategie de test complete, concue a la fois pour garantir la qualite du code et pour servir de support pedagogique dans le cadre de la formation Test/QA. La strategie couvre trois niveaux : les tests unitaires, les tests d'integration et les tests de bout en bout assistes par IA.

7.1 Tests Unitaires

Chaque application Django dispose de son propre fichier de tests unitaires (`tests/test_models.py`) qui valide le comportement des modeles. Les tests verifient notamment : la creation correcte des objets en base de donnees, les contraintes d'integrite (champs uniques, valeurs nulles interdites), les methodes personnalisees (`get_duration_display`, `get_current_price_economy`), et les proprietes calculees. Les tests sont executables via la commande `python manage.py test`.

7.2 Tests d'Integration

Les tests d'integration verifient le bon fonctionnement des interactions entre les differents composants de l'application : vues + modeles + templates + URLs. Ces tests simulent des requetes HTTP completes et valident les reponses (statut 200, contenu attendu, redirections correctes). L'outil Django Test Client est utilise pour ces tests, permettant de simuler le comportement d'un navigateur sans necessite de serveur web actif.

7.3 Tests End-to-End avec IA

Le module `ai_testing` (repertoire `ai_testing/`) implemente un framework de tests de bout en bout assistes par intelligence artificielle. Les fichiers principaux sont `ai_test_tools.py` (outils et utilitaires pour les tests IA) et `tests_e2e.py` (scenarios de test end-to-end). Ce module permet d'automatiser la generation de cas de test, la detection d'anomalies et l'analyse des resultats de test en utilisant des modeles d'intelligence artificielle. Il constitue un element pedagogique important pour la formation, demontrant l'integration de l'IA dans les processus d'assurance qualite.

8. Guide de Deploiement et Installation

8.1 Prerequis

Pour installer et faire fonctionner le projet NouvelAir sur votre machine, les prerequis suivants sont necessaires :

Composant	Version minimale	Note
Python	3.10+	3.12.4 recommandee
pip	Derniere version	Gestionnaire de paquets Python
Navigateur web	Moderne	Chrome, Firefox, Safari ou Edge
Editeur de code	Au choix	VS Code, PyCharm recommandes

8.2 Procedure d'Installation

Etape 1 - Creer l'environnement virtuel (recommande) :

Ouvrez un terminal et executez les commandes suivantes pour creer et activer un environnement virtuel Python isole. Cette pratique est vivement recommandee pour eviter les conflits de dependances avec d'autres projets.

Etape 2 - Installer les dependances :

Executez 'pip install -r requirements.txt' dans le repertoire du projet. Le fichier requirements.txt contient toutes les bibliotheques necessaires : Django, crispy-forms, crispy-bootstrap5, django-countries, django-phonenumbers-field et Pillow. Si vous rencontrez des erreurs d'installation, verifiez que pip est a jour avec 'python -m pip install --upgrade pip'.

Etape 3 - Initialiser la base de donnees :

Executez 'python manage.py makemigrations' puis 'python manage.py migrate' pour creer les tables de la base de donnees SQLite. Ces commandes lisent les fichiers de migration de chaque application et creent les tables correspondantes.

Etape 4 - Peupler la base de donnees :

Executez 'python manage.py populate_data' pour inserer les donnees de test (10 aeroports, 4 appareils, 284 vols, 5 destinations, 3 promotions, 2 utilisateurs). Cette commande peut prendre quelques secondes en raison du volume de donnees generees.

Etape 5 - Lancer le serveur :

Executez 'python manage.py runserver' puis ouvrez votre navigateur a l'adresse <http://127.0.0.1:8000/> pour acceder a l'application.

8.3 Comptes de Test

Identifiant	Mot de passe	Profil	Role
-------------	--------------	--------	------

admin	NouvelAir2025!	Superutilisateur	Administration complete
testuser	NouvelAir2025!	Utilisateur standard	Reservation et navigation

9. Problemes Connus et Solutions

Cette section documente les problemes rencontres lors du developpement et du deploiement initial du projet, ainsi que les solutions appliquees. Elle sert de reference pour les developpeurs et les testeurs qui travaillent sur le projet.

9.1 Erreurs de Namespace URL

Chaque application Django definit un `app_name` dans son `urls.py`, creant un espace de noms pour ses URLs. Les templates doivent imperativement utiliser la syntaxe complete `{% url 'namespace:name' %}` (par exemple `{% url 'flights:home' %}`). L'utilisation de `{% url 'home' %}` sans le prefixe provoque une erreur `NoReverseMatch`. Cette regle s'applique a tous les liens dans les templates, y compris ceux du fichier `base.html` (navbar et footer).

9.2 Champs de Modele

Attention aux noms de champs specifiques lors de l'ecriture de requetes ou de scripts de population. Les erreurs suivantes ont ete rencontrees et corrigees : `Aircraft` utilise `'registration'` (et non `'code'`) et `'model_name'` (et non `'name'`) ; `UserProfile` est accessible via `'user'` (cle etrangere) et non `'username'` ; `Airport` utilise `'latitude'` et `'longitude'` en `FloatField` (NOT NULL, valeurs reelles requises). Ces ecart entre les noms attendus et les noms reels des champs sont une source frequente de bugs et doivent etre verifies avant toute modification du code.

9.3 Filtres de Template

Le filtre de template `'duration_format'` n'existe pas par default dans Django. La solution adoptee a ete de remplacer son utilisation dans les templates par l'appel de la methode de modele `get_duration_display()` directement sur l'objet `Flight`. Par exemple, au lieu de `{{ flight.departure_time|duration_format:flight.arrival_time }}`, utilisez `{{ flight.get_duration_display }}`. Cette methode est definie dans le modele `Flight` et calcule automatiquement la duree a partir des champs `departure_time` et `arrival_time`.

9.4 Pages Legales Manquantes

Le footer de `base.html` contient des liens vers les pages legales (legal) et conditions generales (terms). Si ces pages ne sont pas configurees dans `nouvelair/urls.py`, une erreur `NoReverseMatch`

sera levee a chaque chargement de page. La solution consiste a ajouter les patterns URL suivants dans nouvelair/urls.py : deux chemins pointant vers TemplateView avec les templates legal.html et terms.html, stockes dans le repertoire nouvelair/templates/. L'import de TemplateView depuis django.views.generic est necessaire.

10. Glossaire

Terme	Definition
NoReverseMatch	Erreur Django quand une URL ne peut pas etre resolue depuis un nom
app_name	Espace de noms d'une application Django dans la resolution des URLs
TemplateView	Vue generique Django qui rend un template sans logique metier
Context Processor	Fonction injectant des variables globales dans tous les templates
ORM	Object-Relational Mapping : couche d'abstraction base de donnees de Django
Migration	Fichier de modification du schema de base de donnees versionne
CRUD	Create, Read, Update, Delete - operations de base sur les donnees
TND	Dinar Tunisien - devise officielle de la Tunisie
IATA	Association Internationale du Transport Aerien (codes aeroports)
QA	Quality Assurance - Assurance Qualite